

ENGINEERING COMPARISON

ACI / NAUTOBOT

Nautobot to Cisco ACI Solution Comparison

Platform v2 execution framework versus the standalone ACI Nautobot lab

Comparison date: 28 August 2026

Solution A: C:\MCP\INFRA-AUTOMATION-PLATFORM

Solution B: C:\Claude Code\ACI-Nautobot\nautobot-docker, including its adjacent sales-pbr-nac automation project

Scope: Nautobot data modeling, ACI synchronization/rendering, desired-state artifacts, Terraform execution, validation, governance, auditability, extensibility, and operational cost.

Executive conclusion: Solution A is the stronger platform for repeatable, governed, multi-team automation. Solution B is the simpler and more approachable ACI lab for learning, prototyping, and manually controlled changes. They are complementary in purpose, but should not be treated as equivalent implementations.

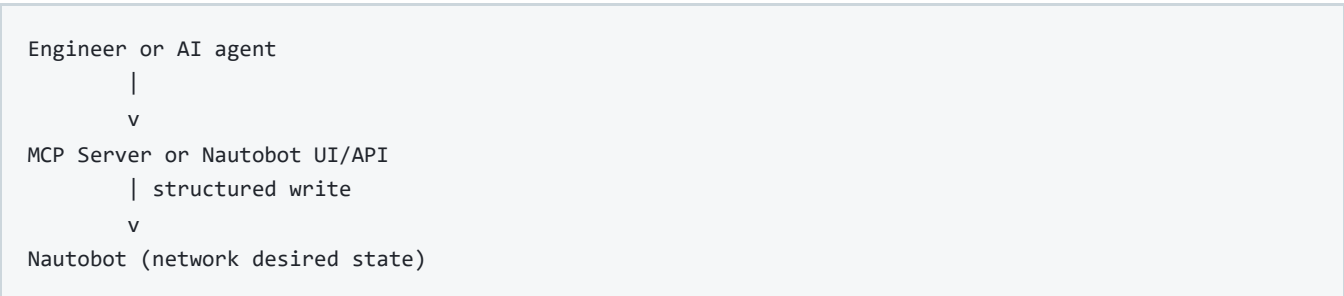
1. Executive Summary

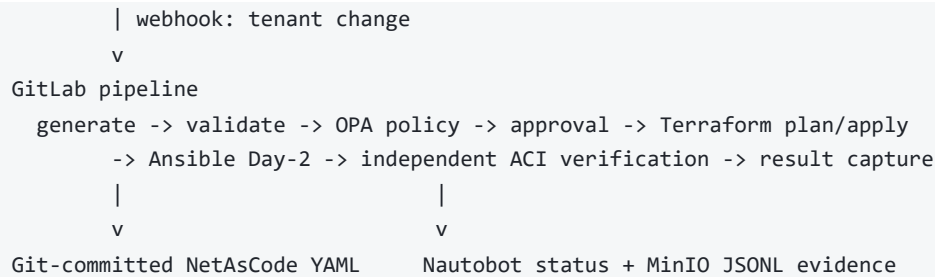
Dimension	Solution A: Platform v2	Solution B: Standalone ACI lab
Primary purpose	Reusable execution framework for ACI and EVPN, with AI/MCP access.	Focused Nautobot + ACI SSoT demonstration and single-lab automation workflow.
Nautobot role	Queryable desired-state origin for network objects; generator produces the Terraform artifact.	Inventory and selected desired-state source; broader policy remains in hand-maintained Git YAML.
ACI path	Nautobot API/GraphQL -> deterministic NetAsCode YAML -> GitLab pipeline -> Terraform/Ansible -> independent verification.	Nautobot seed/UI -> manually run render script -> generated YAML plus manual YAML -> Terraform.
Automation trigger	Nautobot webhook triggers GitLab pipeline.	Operator runs scripts and Terraform manually.
Safety controls	Schema validation, OPA policy, approval gate, plan, apply, post-change verification.	Terraform plan review by the operator.
ACI coverage	Tenants, VRFs, BDs, subnets, EPGs, contracts, filters, L3Out logical objects, access/fabric policy scope.	Core Nautobot-backed objects plus manually maintained policy files; narrower Nautobot model.
Operational cost	High: GitLab, Runner, Vault, MinIO, observability, MCP and multiple networks.	Low: Nautobot, PostgreSQL, Redis, Celery, and an optional toolbox.
Best fit	Controlled team operations and a durable automation platform.	Proof of concept, education, lab experimentation, and low-volume manual operation.

Important distinction: Solution A is not merely a newer renderer. It changes ownership boundaries. Nautobot remains the network Source of Truth, but GitLab owns execution, OPA owns policy decisions, and verification/capture create evidence after execution. Solution B keeps more responsibility with the individual engineer.

2. Architecture and Control Flow

Solution A: Platform v2

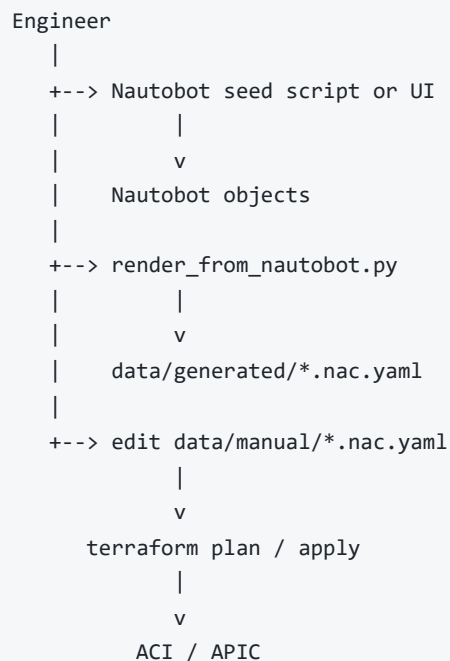




The MCP server exposes narrow business-operation tools such as `create_tenant`, `create_vrf`, `create_bridge_domain`, `create_epg`, `create_contract`, and `create_l3out`. It writes structured objects to Nautobot and reads status back; it is intentionally not the pipeline orchestrator.

Relevant evidence is documented in `knowledge/adr/ADR-016-NetAsCode-Centric-Execution-Framework.md`, `knowledge/architecture/Execution-Framework.md`, `platform/python/generate_aci.py`, `platform/python/generator/client.py`, and `platform/python/generator/transformer.py`.

Solution B: Standalone ACI lab



The standalone project uses Nautobot SSoT ACI capabilities and a separate `sales-pbr-nac` project. Some core values are rendered from Nautobot, while policy-rich configuration remains in manually maintained YAML. The workflow is understandable and direct, but the operator must remember the correct ordering and review boundaries.

3. Nautobot Configuration and Data Model

Area	Solution A	Solution B
Nautobot version posture	Designed as part of the current Platform v2 stack; exact runtime is controlled by the missing independently managed Nautobot checkout in this clone.	Observed local stack uses Nautobot 2.2.9 with Python 3.11 image metadata.
ACI plugin/configuration	ACI automation is primarily implemented in the platform generator and Terraform module. The MCP server uses Nautobot APIs; credentials for ACI execution are read through Vault in the generator/Terraform workflow.	nautobot-ssot is enabled in config/nautobot_config.py. The config also contains environment-based APIC settings for the ACI SSOT job.
Tenant	Nautobot <code>tenancy.Tenant</code> ; ACI namespace and metadata are handled by generator conventions and custom fields.	Nautobot <code>tenancy.Tenant</code> ; seed scripts create the lab tenant and related objects.
VRF	Nautobot <code>ipam.VRF</code> , with ACI-specific custom fields for provider attributes when explicitly set.	Nautobot <code>ipam.VRF</code> , primarily core fields plus seed data.
Bridge Domain	Nautobot <code>ipam.Prefix</code> ; BD identity is derived from the platform's documented prefix convention and custom fields.	Nautobot <code>ipam.Prefix</code> with an <code>aci_bd</code> custom field and seed-script values.
EPG	Nautobot <code>ipam.VLAN</code> with opt-in fields such as <code>aci_application_profile</code> and <code>aci_epg_bridge_domain</code> ; only explicitly marked VLANs become EPGs.	Not modeled as a first-class Nautobot-backed configuration path; policy YAML carries the richer ACI objects.
Contracts / filters / L3Out	Represented through structured tenant custom fields and transformed into the NetAsCode schema; Terraform resources consume the result.	Maintained in manual YAML files rather than being fully represented in Nautobot.
Physical/fabric policy	Logical access/fabric policy coverage exists in the platform design and code; physical bindings remain limited by the ACI simulator.	Policy files can express configuration, but the Nautobot model does not provide the same integrated lifecycle.

Design assessment: Solution A uses Nautobot as a structured input model and extends existing Nautobot objects conservatively with custom fields. Solution B uses a smaller model and deliberately leaves complex policy in Git YAML. That makes B easier to understand initially, but creates two sources of desired-state information.

4. Nautobot-to-ACI Data Path

Solution A: deterministic generator boundary

1. **Read:** `platform/python/generator/client.py` queries Nautobot through GraphQL for tenants, VRFs, prefixes, VLANs, and locations.
2. **Transform:** `platform/python/generator/transformer.py` maps those objects into the ACI NetAsCode structure.
3. **Emit:** `platform/netascode/aci/tenants.yaml` is the technology-specific desired-state artifact consumed by Terraform.
4. **Prove:** the pipeline runs the generator determinism check and compares repeated output.
5. **Execute:** GitLab jobs validate the YAML, evaluate OPA policy, produce a Terraform plan, require approval for apply, then run Ansible and verification.

This boundary is strong because the generator is the only translation point between Nautobot and the ACI artifact. The artifact is reviewable, reproducible, and committed to Git.

Solution B: renderer plus manual policy composition

1. **Seed:** `seed_nautobot.ps1` writes tenants, VRFs, prefixes, devices, and custom fields to Nautobot.
2. **Render:** `sales-pbr-nac/tools/render_from_nautobot.py` reads Nautobot and writes generated files under `data/generated`.
3. **Compose:** hand-maintained files under `data/manual` supply policy not modeled in Nautobot.
4. **Execute:** the operator runs Terraform from the toolbox or host and manually reviews the plan.

This is a good lab workflow when a human is intentionally supervising each step. It is less suitable when the same change must be reproduced by different operators or promoted through environments.

5. Execution, Validation, and Evidence

Capability	Solution A	Solution B
Terraform provider/version discipline	Provider and Terraform versions are explicitly managed in the ACI module and pipeline configuration; provider schemas were validated during implementation.	Terraform workflow is manual and provider pinning is less central to the documented lab flow.
State	Terraform state and plans are handled in the CI execution model with concurrency controls and artifacts.	Local <code>terraform.tfstate</code> ; state handling is primarily the operator's responsibility.
Schema validation	Dedicated <code>validate_nac.py</code> job checks generated YAML before policy and execution.	No equivalent automated validation stage.
Policy	OPA policy job fails closed and blocks execution for violations.	No OPA gate; plan review is the main control.
Approval	GitLab manual apply gate, with protected-branch behavior in the Community Edition lab.	Human chooses when to run <code>terraform apply</code> .
Post-change verification	ACI REST/pyATS-style verification runs independently from Terraform and writes status back to Nautobot.	Manual APIC GUI or API inspection.
Knowledge/audit capture	GitLab logs and artifacts, Nautobot status fields, and MinIO JSONL deployment records.	Git history and local Terraform state; no integrated execution record.
Day-2 automation	Ansible playbooks are a defined pipeline stage after provisioning.	No comparable integrated Day-2 stage in the standalone workflow.

6. Pros and Cons

Solution A: Platform v2

Pros	Cons
• End-to-end repeatability from Nautobot change to verified execution. • Clear separation of desired state, execution, policy, approval, and observed state. • Strong governance: OPA and approval stop unsafe changes before apply. • Generated YAML is deterministic, reviewable, and Git-committed. • Supports richer ACI coverage and a second domain, EVPN, using the same mechanism. • MCP tools allow AI clients to request structured operations without direct APIC access. • Independent verification and knowledge capture make failures diagnosable.	• High memory and CPU footprint; the documented full lab is approximately 26 GB RAM and 14 vCPUs. • Many services and credentials must be initialized correctly. • GitLab Omnibus startup and runner behavior add operational complexity. • Failure diagnosis spans Nautobot, webhooks, GitLab, OPA, Vault, Terraform, Ansible, and APIC. • Physical ACI bindings and L3Out routing are limited by the simulator, not by the software model. • The current clone is incomplete because its independently managed Nautobot checkout is absent.

Solution B: Standalone ACI lab

Pros	Cons
• Small footprint and straightforward local startup. • Easy to understand for a single engineer learning Nautobot SSoT and ACI. • All core infrastructure runs in one focused Compose stack. • Manual policy YAML gives an operator immediate access to provider capabilities without first modeling every object in Nautobot. • Useful for experimentation and controlled proof-of-concept work.	• Manual sequencing is easy to forget or perform out of order. • Nautobot and manual Git YAML jointly hold desired state, creating split ownership. • No automated policy, approval, independent verification, or knowledge capture. • Multi-tenant and broader domain expansion require script and file changes. • Reproducibility depends on operator discipline and local environment state. • Credentials are more closely tied to environment files and SSoT configuration; the documented SSoT environment variables may be deprecated in newer plugin releases. • Limited CI/CD and AI-client integration.

7. Resource and Operational Considerations

Concern	Impact on Solution A	Impact on Solution B
RAM/CPU	Large. GitLab is the dominant consumer; the full stack is not a comfortable fit for a 16 GB laptop.	Moderate. The existing Nautobot stack already consumes meaningful RAM, but is materially lighter than the full platform.
Ports	Requires coordinated ports for Nautobot, GitLab, Vault, MCP, OPA, MinIO, and observability.	Primarily Nautobot at port 8080 plus internal service ports.
Persistent data	Many named volumes: GitLab, Vault, runner, MinIO, observability, and Nautobot data.	Focused PostgreSQL, Redis, media, and static volumes.
Recovery	Documented recovery process, but more services and secrets must be restored together.	Fewer components and simpler recovery, though local state and manual files still need backup.
Change blast radius	Pipeline controls reduce accidental apply, but a bad object can affect the domain-wide generated artifact.	Smaller system, but a human can bypass any absent guardrail by applying manually.

8. Recommendation

For a production-like platform or team workflow, choose Solution A. Its additional complexity buys controlled execution, policy enforcement, auditability, repeatability, and a clean path to additional domains and AI clients.

For a laptop-based ACI learning or development environment, choose Solution B. It is substantially lighter and gives an engineer a direct view of Nautobot, rendered YAML, and Terraform. It should be treated as a lab/prototype rather than the governance model for shared infrastructure.

Recommended relationship between them: use the standalone project as a lightweight ACI development or data-modeling sandbox, but promote proven patterns into Platform v2 rather than connecting both projects to the same live APIC without explicit state ownership. Do not let both Terraform roots manage the same ACI objects or use the same Terraform state.

Most important operational rule: one ACI object must have one authoritative Terraform state and one clearly owned desired-state path. Running both solutions against the same tenant, VRF, bridge domain, or policy without strict separation risks drift and competing changes.

9. Source Evidence and Assumptions

- Solution A documentation: README.md, Platform-Administration-Guide.md, knowledge/architecture/Platform-v2-Reference-Architecture.md, knowledge/architecture/Execution-Framework.md, knowledge/architecture/Platform-Status-and-Pending-Items.md, and ADR-016 through ADR-021.

- Solution A implementation: `platform/python/generate_aci.py`, `platform/python/generator/client.py`, `platform/python/generator/transformer.py`, `platform/terraform/aci/`, `platform/ansible/aci/`, `pipelines/`, and `mcp-server/src/mcp_server/`.
- Solution B implementation: `C:\Claude Code\ACI-Nautobot\nautobot-docker\config\nautobot_config.py`, its Compose file, seed scripts, and the adjacent `sales-pbr-nac-renderer/data/workflow` files.
- The current checkout does not contain `docker/nautobot/`; Solution A's root Compose file references that independently managed checkout. This comparison therefore evaluates the documented and available Platform v2 source, while noting that its full local stack cannot be recreated from this clone alone.
- Exact runtime resource usage varies with data volume, Docker Desktop settings, GitLab activity, and whether the ACI simulator/APIC is reachable.

Prepared from read-only inspection of both local solutions on 28 August 2026. This report compares design and implementation evidence; it does not modify either project or any running container.